Facultad de Ingenierías

PROGRAMACIÓN DE SISTEMAS WEB

Actividad Integradora 2

Lista de tareas: gestor de tareas con JavaScript Vanilla, API REST y persistencia con localStorage

Tema de investigación: Opción A — Arquitectura Modular ES6

Estudiante: Angel Mauricio Guevara Chinchero

Carrera: Ingeniería en Sistemas Inteligentes

Docente: Gregorio Cruz Gonzabay

Repositorio: https://github.com/aguevaranv/webdev_act_02

Fecha de entrega: 01 de noviembre de 2026

1. Introducción

Lista de Tareas es una aplicación web para gestionar tareas, desarrollada con HTML, CSS y **JavaScript Vanilla.** Permite crear, editar, eliminar y marcar tareas como completadas; filtrarlas por estado; buscarlas por título en tiempo real; ordenarlas por fecha o prioridad, y ver un panel de métricas que se actualiza con cada cambio. La primera vez que se abre, importa tareas de ejemplo desde la API REST pública **JSONPlaceholder** y, a partir de ahí, guarda todo en el navegador con `localStorage`, por lo que también funciona sin conexión a internet.

2. Resumen de la investigación: Arquitectura Modular ES6

2.1 ¿Qué es un módulo?

Un módulo ES6 es un archivo JavaScript con **su propio ámbito**: las variables y funciones que declara son privadas y no se mezclan con las de otros archivos. Para compartir algo, el módulo lo **exporta** con `export`, y el archivo que lo necesita lo **importa** con `import`. Antes de ES6 (2015), todos los scripts de una página compartían el mismo espacio global, lo que provocaba choques de nombres y dependencias difíciles de seguir.

2.2 Cómo se usa en el proyecto

Se usan **exportaciones con nombre**. Por ejemplo, `storage.js` exporta tres funciones y mantiene privada la clave con la que guarda los datos:

```
// storage.js
const CLAVE = 'taskmanager_tareas';          // privada: no se exporta
export function cargarTareas() { ... }
export function guardarTareas(tareas) { ... }
export function hayDatosGuardados() { ... }

// app.js
import { cargarTareas, guardarTareas, hayDatosGuardados } from './storage.js';
```

En el HTML solo se enlaza el archivo principal con `<script type="module" src="js/app.js">` y el navegador carga el resto siguiendo los `import`.

2.3 Características investigadas

- **Ámbito propio:** nada es global salvo lo que se exporta, así se evitan choques de nombres.
- **Carga diferida:** un script con `type="module"` se ejecuta cuando el HTML ya está listo (como `defer`), por lo que los elementos del DOM ya existen.
- **Modo estricto y una sola evaluación:** siempre se ejecutan en `'use strict'` y, aunque varios archivos importen el mismo módulo, su código corre una sola vez.
- **Necesitan un servidor:** abiertos con doble clic (`file://`) el navegador los bloquea por la política CORS. Durante el desarrollo se usó la extensión Live Server de VS Code.
- **Rutas explícitas:** el `import` debe incluir `./` y la extensión `.js`; de lo contrario aparece el error `Failed to resolve module specifier`.

3. Arquitectura del proyecto y razón de la distribución

3.1 Estructura de carpetas

```
webdev_act_02/
├── index.html        Estructura de la página
├── css/
│   └── styles.css    Estilos (variables, flexbox, responsive)
└── js/
    ├── app.js        Punto de entrada: estado, eventos e inicio
    ├── ui.js         Interfaz visual (DOM)
    ├── tareas.js     Lógica de las tareas (datos puros)
    ├── storage.js    Gestor de localStorage
    └── api.js        Servicio de la API REST
```

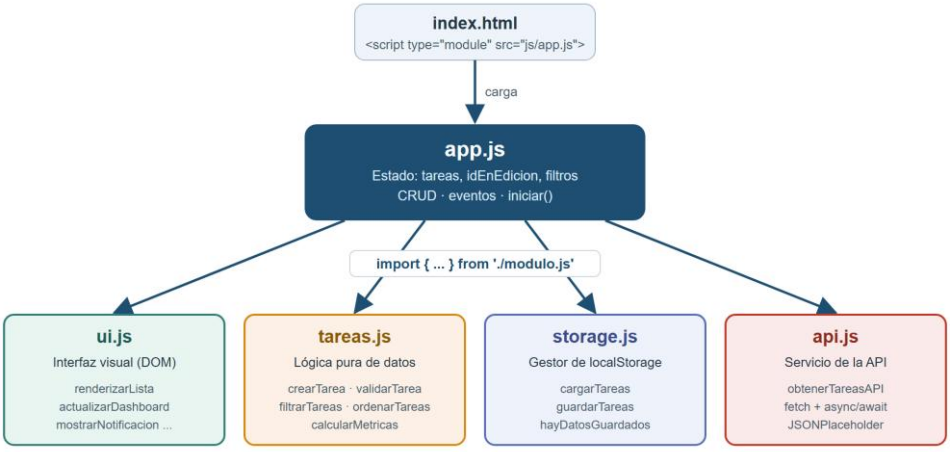


Figura 1. Dependencias entre módulos: app.js importa a los otros cuatro, que no se importan entre sí.

3.2 Responsabilidad de cada módulo

Módulo	Responsabilidad	Exporta
`app.js`	Coordina todo: guarda el estado (tareas, tarea en edición, filtros), escucha los eventos y arranca la app.	Nada (es el punto de entrada)
`ui.js`	Todo lo que toca el DOM: dibujar la lista, el dashboard, notificaciones y el formulario.	`renderizarLista`, `actualizarDashboard`, `mostrarNotificacion`, `mostrarCargando`, `leerFormulario`, `cargarEnFormulario`, `limpiarFormulario`...
`tareas.js`	Reglas de negocio sin DOM ni almacenamiento: recibe datos y devuelve resultados.	`crearTarea`, `validarTarea`, `filtrarTareas`, `ordenarTareas`, `calcularMetricas`
`storage.js`	Leer y escribir en localStorage con JSON.	`cargarTareas`, `guardarTareas`, `hayDatosGuardados`
`api.js`	Pedir tareas a JSONPlaceholder y convertirlas al formato de la app.	`obtenerTareasAPI`

3.3 ¿Por qué se distribuyó así?

La división sigue el principio de **separación de responsabilidades**: cada archivo responde a una sola pregunta. La actividad pedía separar la interfaz, el servicio de la API y el gestor de localStorage, y a esas tres capas se agregaron dos más:

- **tareas.js como lógica pura.** Filtrar, ordenar, validar y contar no necesitan saber nada de la página. Al no tocar el DOM ni el almacenamiento, estas funciones se pueden probar y reutilizar solas; por ejemplo, `ordenarTareas` trabaja sobre una copia (`[...tareas]`) para no alterar el arreglo original.
- **app.js como coordinador.** Es el único que conoce a todos los demás. Así, si mañana los datos se guardaran en un servidor PHP en lugar de localStorage, solo cambiaría `storage.js`, y el resto del código seguiría igual.
- **Sin dependencias circulares.** Los botones de cada tarea (completar, editar, eliminar) se crean en `ui.js`, pero las funciones que modifican el estado están en `app.js`. Para que `ui.js` no tenga que importar `app.js` (y viceversa), `app.js` le pasa un objeto `acciones` como parámetro de `renderizarLista`. De esta forma, `ui.js` sabe dibujar pero no decide qué hacer con los datos.
- **Edición separada en estado y vista.** Al editar, `app.js` solo guarda el id de la tarea (`idEnEdicion`), y `ui.js` se encarga de lo visual con `cargarEnFormulario` y `limpiarFormulario`.

4. Persistencia con localStorage y carga desde la API

Las tareas se guardan bajo la clave `taskmanager_tareas`. Como localStorage solo almacena texto, el arreglo se convierte con `JSON.stringify()` al guardar y con `JSON.parse()` al leer. Cada acción (crear, editar, eliminar o completar) llama a `guardarYActualizar()`, que guarda y vuelve a dibujar la vista.

Al iniciar, la función asíncrona `iniciar()` revisa `hayDatosGuardados()`. Solo si localStorage está vacío consulta la API con `fetch` y `async/await`, convierte las tareas recibidas e **importa únicamente las que no existen**

(comparando ids con `some()`), por lo que recargar la página nunca duplica registros. Si la red o el servidor fallan, `try/catch` captura el error y se muestra una notificación roja en pantalla; la aplicación sigue funcionando con los datos locales.

5. Evidencias

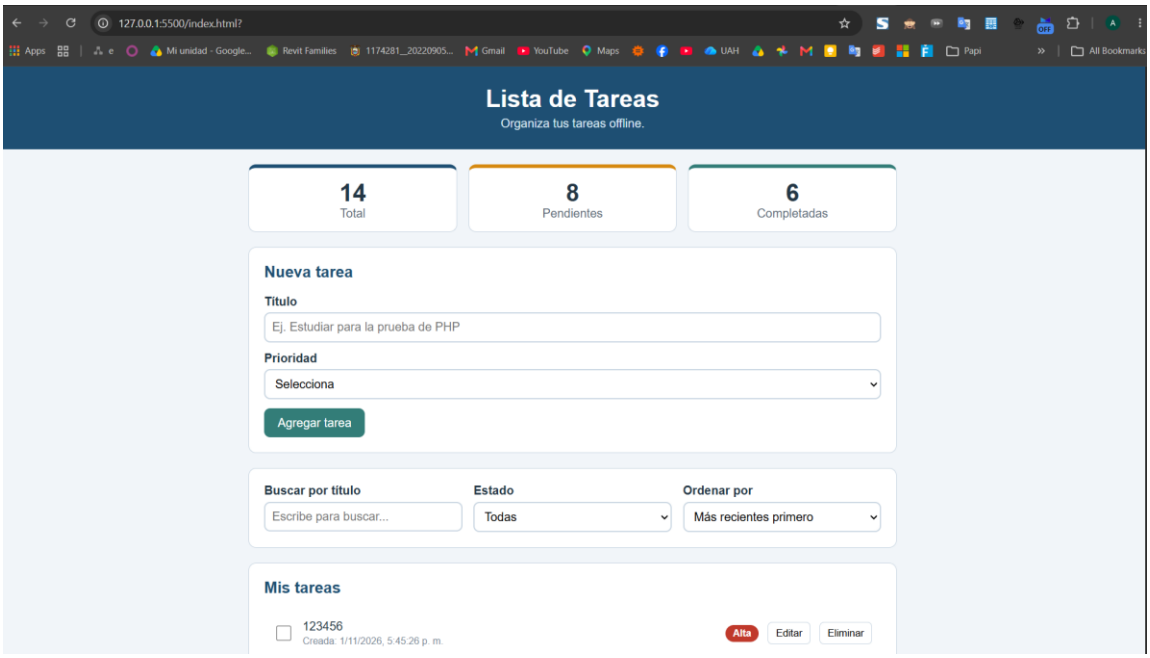


Figura 2. Lista de Tareas funcionando: dashboard, formulario, filtros y lista con las tareas importadas desde la API.

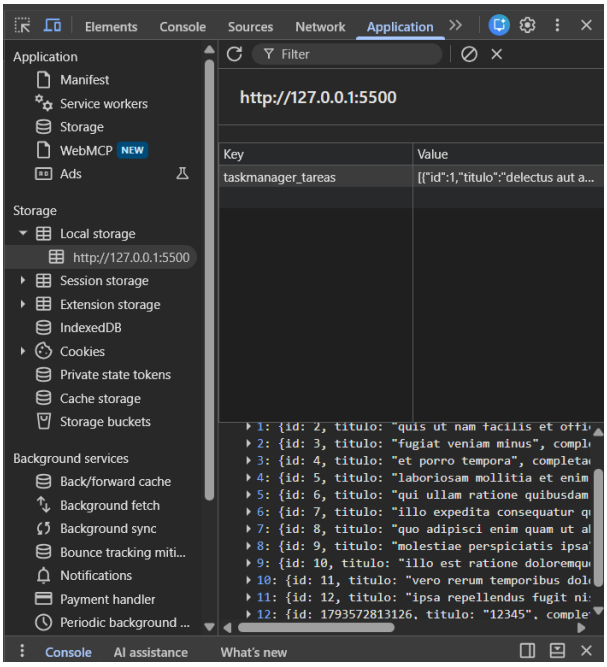


Figura 3. Application > Local Storage con la clave taskmanager_tareas y las tareas guardadas.

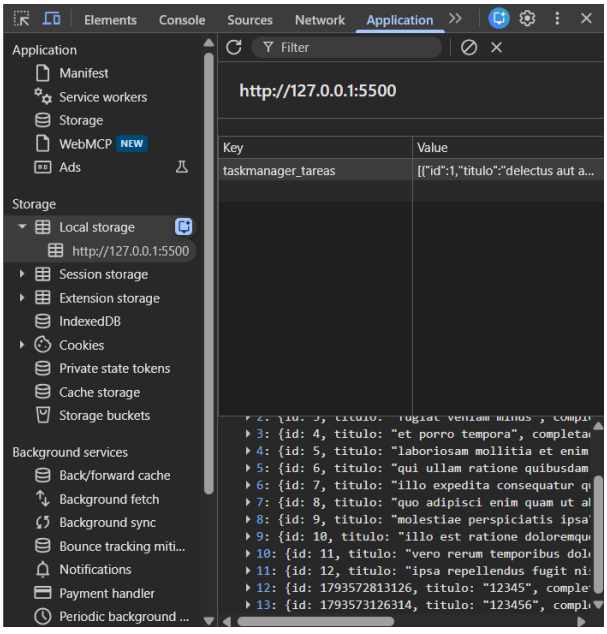


Figura 4. La tarea creada permanece en Local Storage después de recargar la página.

6. Conclusión

Organizar La lista de Tareas en módulos ES6 permitió que cada parte del código tuviera un propósito claro: la interfaz, la lógica de las tareas, el almacenamiento y la API quedaron separados y conectados solo a través de `import` y `export`. Esto hizo más fácil encontrar errores, ya que cada problema apuntaba a un solo archivo, y deja la aplicación preparada para crecer, por ejemplo reemplazando localStorage por un servidor propio sin reescribir la interfaz.